

Διάλεξη 06/07 – Τοπικό Σύστημα Αρχείων

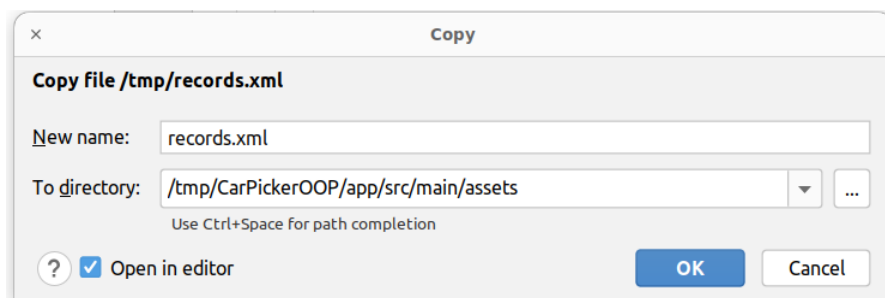
Στόχοι της εφαρμογής είναι:

- Να διαβάζουμε τα δεδομένα μας που βρίσκονται αποθηκευμένα σε αρχείο xml από το τοπικό σύστημα αρχείων χρησιμοποιώντας ResourceManager.
- Να επεξεργαστούμε (parse) το xml αρχείο.
- Να καταγράφουμε στην ενσωματωμένη στο Android βάση δεδομένων Sqlite τις ενέργειές μας.

Θα χρησιμοποιήσουμε το ολοκληρωμένο project της προηγούμενης εβδομάδας και θα δουλέψουμε πάνω σε αυτό. Επιπλέον, προσέχουμε ώστε το API Level να είναι τουλάχιστον 26 ώστε να χρησιμοποιήσουμε τον ενσωματωμένο DB Inspector (**View > Tool Windows > App Inspection > Database Inspector**).

Κάνουμε δεξί κλικ πάνω στο project (app) και επιλέγουμε New > Folder > Assets. Στο παράθυρο διαλόγου που θα εμφανιστεί επιλέγουμε Finish, χωρίς να τροποποιούμε κάτι. Ο φάκελος αυτός θα χρησιμοποιηθεί από τον AssetsManager για να διαβάσει το xml αρχείο που θα χρησιμοποιήσουμε.

Το αρχείο αυτό βρίσκεται στο openeclass στην τρέχουσα διάλεξη και ονομάζεται records.xml. Το κατεβάζουμε τοπικά, κάνουμε αντιγραφή και επικόλληση μέσα από το IDE στον φάκελο Assets. Στο παράθυρο διαλόγου που θα εμφανιστεί, επιλέγουμε OK



Βλέπουμε ότι έχει ανοίξει στον editor το αρχείο xml με τα παρακάτω περιεχόμενα:

```
<?xml version="1.0"?>
<records>
  <carBrand>
    <name>Toyota</name>
    <models> Yaris, Auris, Corolla </models>
  </carBrand>

  <carBrand>
    <name>VW </name>
    <models> Golf, Polo </models>
  </carBrand>

  <carBrand>
    <name>Nissan</name>
    <models> Sunny </models>
  </carBrand>
</records>
```

1. Συγγραφή κώδικα για ανάγνωση δεδομένων από το αρχείο XML.

Πλέον, μπορούμε να αρχίσουμε να γράφουμε τον κώδικά μας. Θα δημιουργήσουμε έναν νέο κατασκευαστή στην κλάση CarBrandsList ο οποίος θα παίρνει ως παράμετρο τον AssetManager. Όταν χρησιμοποιούμε αυτόν τον κατασκευαστή, η εφαρμογή μας θα διαβάζει δεδομένα από το XML αρχείο. Ο κώδικας που θα προσθέσουμε θα είναι ο παρακάτω, στον οποίο πραγματοποιείται και επεξεργασία του xml αρχείου:

```
public CarBrandsList(AssetManager assets) {

    try {
        InputStream is = assets.open("records.xml");

        DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
        DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
        Document doc = dBuilder.parse(is);
        Element element=doc.getDocumentElement();
        element.normalize();

        NodeList nList = doc.getElementsByTagName("carBrand");

        for (int i=0; i<nList.getLength(); i++) {

            Node node = nList.item(i);
            if (node.getNodeType() == Node.ELEMENT_NODE) {
                NodeList nameNode = ((Element)
node).getElementsByTagName("name").item(0).getChildNodes();
                String brand = nameNode.item(0).getNodeValue();
                NodeList modelsNode = ((Element)
node).getElementsByTagName("models").item(0).getChildNodes();
                String models = modelsNode.item(0).getNodeValue();
                bList.add(new Brand(brand, models));
            }
        }

    } catch (Exception e) {e.printStackTrace();}
}
```

Παρατηρούμε ότι χρειαζόμαστε έναν νέο κατασκευαστή για το Brand, ο οποίος θα παίρνει 2 παραμέτρους: Το brand, το οποίο είναι το όνομα του κατασκευαστή και το models το οποίο είναι μία λίστα με τα μοντέλα χωρισμένα με κόμματα. Προσθέτουμε τον εν λόγω κατασκευαστή στην κλάση Brand:

```
public CarBrand(String b, String m) {
    name = b;
    models = Arrays.asList(m.split(",", -1));
    System.out.println(models);
}
```

και αλλάζουμε το

```
private ArrayList<String> mList = new ArrayList<String>();
```

σε

```
private List<String> mList = new ArrayList<String>();
```

Στη MainActivity (PickCarActivity) πλέον, αντικαθιστούμε τον υπάρχοντα κατασκευαστή για την με τον παρακάτω, ώστε να παίρνουμε τα δεδομένα από το αρχείο xml:

```
//cbList = new CarBrandsList();
cbList = new CarBrandsList(getAssets());
```

2. Συγγραφή κώδικα για αποθήκευση δεδομένων στην SQLite.

Δημιουργούμε μία νέα κλάση Java που λέγεται SelectionLoggerContract. Σε αυτή, προσθέτουμε τον παρακάτω κώδικα που περιλαμβάνει τα πεδία του πίνακα:

```
public final class SelectionLoggerContract {
    private SelectionLoggerContract(){};

    public static class LoggerEntry implements BaseColumns{
        public static final String TABLE_NAME = "selections";
        public static final String COLUMN_NAME_MODEL = "model";
        public static final String COLUMN_NAME_BRAND = "brand";
        public static final String COLUMN_NAME_TIMESTAMP = "timestamp";
    }
}
```

Δημιουργούμε ακόμη μία νέα κλάση Java που λέγεται SelectionLoggerDbHelper. Σε αυτή, προσθέτουμε τον παρακάτω κώδικα που περιλαμβάνει τα queries για τη δημιουργία του πίνακα:

```
public class SelectionLoggerDbHelper extends SQLiteOpenHelper {
    public static final int DATABASE_VERSION = 1;
    public static final String DATABASE_NAME = "SelectionLogger.db";

    private static final String SQL_CREATE_ENTRIES =
        "CREATE TABLE " + SelectionLoggerContract.LoggerEntry.TABLE_NAME + " (" +
        SelectionLoggerContract.LoggerEntry._ID + " INTEGER PRIMARY KEY," +
        SelectionLoggerContract.LoggerEntry.COLUMN_NAME_BRAND + " TEXT," +
        SelectionLoggerContract.LoggerEntry.COLUMN_NAME_MODEL + " TEXT," +
        SelectionLoggerContract.LoggerEntry.COLUMN_NAME_TIMESTAMP + " TEXT)";

    public SelectionLoggerDbHelper(Context context){
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase sqLiteDatabase) {
        sqLiteDatabase.execSQL(SQL_CREATE_ENTRIES);
    }

    @Override
    public void onUpgrade(SQLiteDatabase sqLiteDatabase, int i, int i1) {
        sqLiteDatabase.execSQL(SQL_DELETE_ENTRIES);
        onCreate(sqLiteDatabase);
    }
}
```

Τέλος, δημιουργούμε μία νέα κλάση Java που λέγεται SQLiteConnection. Σε αυτή, προσθέτουμε τον παρακάτω κώδικα που διατηρεί τις ρυθμίσεις της σύνδεσης και το query με το οποίο πραγματοποιεί εισαγωγή:

```
public class SQLiteConnection {
    private SelectionLoggerDbHelper dbHelper;
    private SQLiteDatabase db;

    SQLiteConnection(Context context){
        dbHelper = new SelectionLoggerDbHelper(context);
        db = dbHelper.getWritableDatabase();
    }

    public long insert(String brand, String model, String timestamp){

        ContentValues values = new ContentValues();
        values.put(SelectionLoggerContract.LoggerEntry.COLUMN_NAME_BRAND, brand);
```

```

        values.put(SelectionLoggerContract.LoggerEntry.COLUMN_NAME_MODEL, model);
        values.put(SelectionLoggerContract.LoggerEntry.COLUMN_NAME_TIMESTAMP, new
Date(System.currentTimeMillis()).toString());

        long newRowId = db.insert(SelectionLoggerContract.LoggerEntry.TABLE_NAME, null,
values);
        return newRowId;

    }

    public void close() {
        db.close();
    }
}

```

Στην κλάση PickCarActivity προσθέτουμε τον ανάλογο κώδικα για χρήση της παραπάνω κλάσης:
 Προσθέτω ένα γνώρισμα στην κλάση τύπου SQLiteConnection:
 SQLiteConnection lite;

και το αρχικοποιώ στην onCreate:

```
lite = new SQLiteConnection(this);
```

Τέλος, αλλάζω και τον OnCheckedChangeListener:

```

public void onCheckedChanged(RadioGroup group, int checkedId) {
    RadioButton rb=(RadioButton)findViewById(checkedId);
    Toast.makeText(getApplicationContext(), "Number of row inserted: "+lite.insert(brand,
rb.getText().toString(), "just"), Toast.LENGTH_SHORT).show();
}

```

S